

RELATÓRIO — Regressão e correção: ficha e lanterna (MRP2026 Check-In)

Data: 26/09/2026 · Base: v0.7.0 + dark theme + paisagem + modo dev · Metodologia: regressão primeiro, fix depois; sem testes de teatro — só o que roda de verdade neste projeto.

1. Linha de base (antes do fix)

Suíte JUnit4 pura (scripts/test.sh): **39 testes OK** em 9 classes (core, net, scan, DevFixtures). Build CLI (aapt2+javac+D8) OK e apksigner verify OK nos dois APKs. Nenhum teste cobria a ficha nem a política da lanterna — lacuna real, não coberta por teatro.

2. Hipótese 1 — botões encostando: VERDADEIRA

Evidência: ResultSheet.java criava presenceButton, conformityButton e rescaneButton via M3.filledButton/tonalButton — fábricas que devolvem Button sem nenhuma margem (M3.java, baseButton) — e adicionava os três direto no LinearLayout vertical. No fluxo veículo, conformityButton + rescaneButton ficam VISIBLE adjacentes: 0dp de separação. Correção: constante RESULT_BUTTON_GAP_DP=12 e LayoutParams com topMargin de 12dp no botão reserva (cobre os dois fluxos, pois presença e conformidade são mutuamente exclusivos).

3. Hipótese 1b — ✓ verde 'peludo': VERDADEIRA (depende de fonte/DPI)

Evidência: M3.successBlock desenhava o check como glifo de texto '\u2713' a 56sp bold — rasterização da fonte do aparelho, sem anti-alias garantido; em algumas densidades fica serrilhado. Correção: nova View CheckBadge (ui/CheckBadge.java) que desenha o check em vetor com Paint.ANTI_ALIAS_FLAG, stroke round 14% da menor dimensão, 72dp centralizado. Sem dependência nova, funciona no light e no dark (cor via onSuccessContainer).

4. Hipótese 2 — lanterna some no QR de veículo: PARCIALMENTE VERDADEIRA

O sumiço ao abrir a ficha é **by design**: updateTorchVisibility() esconde a lanterna sempre que a ficha está visível (ScanActivity). O bug real estava no wiring, em dois pontos: (a) onFlashSupport() forçava VISIBLE/GONE direto, ignorando a ficha — se o callback da câmera disparasse com a ficha aberta, a lanterna saltava por cima da ficha (ou sumia fora de hora); (b) onResume() reiniciava a câmera sem reavaliar a lanterna — voltando do Check (fluxo veículo), o estado podia ficar preso em GONE. Correção: onFlashSupport agora delega a updateTorchVisibility(); onResume chama updateTorchVisibility(); política extraída para torchShouldShow(sheetVisible, flashAvailable) pura e coberta por 4 testes JVM.

5. Testes (reais, rodam aqui)

Novo: ScanActivityTorchPolicyTest — matriz 2x2 da política (ficha aberta/fechada × flash sim/não). Resultado após o fix: **43 testes OK, 0 falhas** (39 + 4). Verificação de build: build.sh OK, build-dev.sh OK, apksigner verify OK nos dois APKs (release cf2552ee..., dev 72a6d687...). O que NÃO dá para testar sem aparelho/emulador: pixels (gap visual, suavidade do vetor) e ciclo de vida real da câmera — cobertos pelo protocolo manual §7.

6. Workflow do Akita — cabe no projeto?

Foto avaliada: Screenshot_20260922_184738_YouTube.jpg (akitaonrails.com) — pipeline Commit → Testes → CI multiplataforma → Feature flag → Staging → Produção, com reversão rápida e promoção de uso real a issue/PR.

Camada	Cabe aqui?	Estado
--------	------------	--------

Commit (dev/agente escreve o diff)	Sim	Em uso
Testes automatizados	Sim	Em uso (43 JUnit); faltam UI/Robolectric (pós-Gradle)
CI multiplataforma	Parcial	Ainda manual no Termux; propor GitHub Actions no passo Gradle
Feature flag	Sim	Em uso na prática: modo dev (dev://offline / pacote .dev)
Staging (smoke antes do real)	Adaptado	APK dev lado a lado = staging; release = produção
Produção monitorada + rollback	Parcial	crash.log local + mesmo keystore (update por cima); sem telemetria
Uso real vira issue/PR	Sim	Adotar: todo bug de campo abre item com protocolo §7

Veredito: o esqueleto cabe e já opera em versão mínima (dev=staging, release=produção, flags, testes). Faltam CI e mitigação por modo de falha — entram com a migração Gradle.

7. Protocolo manual no aparelho (sem fotos neste relatório)

Prints: impossíveis neste box — sem adb, emulador ou scrpy (which/adb vazio, sem ANDROID_SDK). Validação fica no aparelho real com o MRP Dev instalado lado a lado: (1) Simular QR veículo → conformidade e rescan com respiro de 12dp; (2) bloco ✓ liso em mdpi/xhdpi, light e dark; (3) ficha aberta esconde lanterna; fechar ficha (back) mostra de novo; (4) veículo → Conformidade → voltar: ficha restaurada, lanterna escondida com ficha e visível sem ficha; (5) aparelho sem flash: botão oculto; (6) comparar antes/depois com o release.

8. Arquivos alterados

```
src/mrp/checkin/ui/ResultSheet.java – gap 12dp (RESULT_BUTTON_GAP_DP)
src/mrp/checkin/ui/CheckBadge.java – NOV0, check vetorial anti-aliased
src/mrp/checkin/ui/M3.java – successBlock usa CheckBadge
src/mrp/checkin/ScanActivity.java – torchShouldShow + onFlashSupport/onResume via política
tests/src/mrp/checkin/ScanActivityTorchPolicyTest.java – NOV0, 4 testes
scripts/test.sh – registra a nova classe
```

Evidência de execução: test.sh 43 OK · build.sh OK · build-dev.sh OK · verify OK (release + dev) · APKs em ~/storage/downloads.